In [1]:
```
# Import libs
import numpy as np
import pandas as pd
from backtesting import Strategy, Backtest
```

```
c:\Users\Ismail\AppData\Local\Programs\Python\Python311\Lib\site-packages\backtesting\_plotting.py:55: UserWarning: Jupyter Notebook detected. Setting Bokeh output to notebook. This may not work in Jupyter clients without JavaScript support, such as old IDEs. Reset with `backtesting.set_bokeh_output(notebook=False)`.
  warnings.warn('Jupyter Notebook detected. '
```



BokehJS 3.8.2 successfully loaded.

In [2]:
```
# Read CSV minute data
working_data = pd.read_csv('df_1min.csv', parse_dates=['date'], low_memory=False)
working_data
```

Out[2]:

| | date | Open | High | Low | Close | Volume |
|---|---|---|---|---|---|---|
| **0** | 2010-01-03 12:00:00-05:00 | 1865.50 | 1867.50 | 1865.50 | 1867.00 | 251 |
| **1** | 2010-01-03 12:01:00-05:00 | 1866.75 | 1867.00 | 1866.50 | 1866.50 | 76 |
| **2** | 2010-01-03 12:02:00-05:00 | 1866.75 | 1867.50 | 1866.75 | 1866.75 | 151 |
| **3** | 2010-01-03 12:03:00-05:00 | 1866.75 | 1867.00 | 1866.75 | 1867.00 | 26 |
| **4** | 2010-01-03 12:04:00-05:00 | 1866.75 | 1866.75 | 1866.75 | 1866.75 | 1 |
| **...** | ... | ... | ... | ... | ... | ... |
| **5465281** | 2026-06-25 23:07:00-04:00 | 29237.75 | 29240.75 | 29220.50 | 29230.25 | 212 |
| **5465282** | 2026-06-25 23:08:00-04:00 | 29233.25 | 29244.75 | 29225.75 | 29237.25 | 194 |
| **5465283** | 2026-06-25 23:09:00-04:00 | 29237.25 | 29239.50 | 29224.75 | 29228.25 | 163 |
| **5465284** | 2026-06-25 23:10:00-04:00 | 29224.00 | 29253.75 | 29215.25 | 29253.75 | 290 |
| **5465285** | 2026-06-25 23:11:00-04:00 | 29251.75 | 29254.25 | 29245.00 | 29248.50 | 136 |

5465286 rows × 6 columns

In [3]:
```
# Create new dataframe
df = working_data.copy()
```

In [4]:
```
# ── Volume Delta (1-minute bars) ──────────────────────────────────────
# Uses the candle body direction to split volume into buy/sell pressure.
# Formula: delta = buy_vol - sell_vol
# where buy_vol  = Volume * ((Close - Low)  / (High - Low))
#       sell_vol = Volume * ((High - Close) / (High - Low))

def calculate_delta(df):
    df = df.copy()
```

```
    hl_range = (df["High"] - df["Low"]).mask(lambda s: s == 0)

    buy_vol = df["Volume"] * ((df["Close"] - df["Low"]) / hl_range)
    sell_vol = df["Volume"] * ((df["High"] - df["Close"]) / hl_range)

    df["delta"] = (buy_vol - sell_vol).fillna(0.0)

    return df

df = calculate_delta(df)
```

In [5]:
```
# Show df
df
```

Out[5]:

| | date | Open | High | Low | Close | Volume | delta |
|---|---|---|---|---|---|---|---|
| **0** | 2010-01-03 12:00:00-05:00 | 1865.50 | 1867.50 | 1865.50 | 1867.00 | 251 | 125.500000 |
| **1** | 2010-01-03 12:01:00-05:00 | 1866.75 | 1867.00 | 1866.50 | 1866.50 | 76 | -76.000000 |
| **2** | 2010-01-03 12:02:00-05:00 | 1866.75 | 1867.50 | 1866.75 | 1866.75 | 151 | -151.000000 |
| **3** | 2010-01-03 12:03:00-05:00 | 1866.75 | 1867.00 | 1866.75 | 1867.00 | 26 | 26.000000 |
| **4** | 2010-01-03 12:04:00-05:00 | 1866.75 | 1866.75 | 1866.75 | 1866.75 | 1 | 0.000000 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **5465281** | 2026-06-25 23:07:00-04:00 | 29237.75 | 29240.75 | 29220.50 | 29230.25 | 212 | -7.851852 |
| **5465282** | 2026-06-25 23:08:00-04:00 | 29233.25 | 29244.75 | 29225.75 | 29237.25 | 194 | 40.842105 |
| **5465283** | 2026-06-25 23:09:00-04:00 | 29237.25 | 29239.50 | 29224.75 | 29228.25 | 163 | -85.644068 |
| **5465284** | 2026-06-25 23:10:00-04:00 | 29224.00 | 29253.75 | 29215.25 | 29253.75 | 290 | 290.000000 |
| **5465285** | 2026-06-25 23:11:00-04:00 | 29251.75 | 29254.25 | 29245.00 | 29248.50 | 136 | -33.081081 |

5465286 rows × 7 columns

In [6]:
```
# Make sure date is datetime format
df["date"] = pd.to_datetime(
    df["date"],
    utc=True,
    errors="coerce"
)
```

```
# Make sure date is New York time zone
df["date"] = df["date"].dt.tz_convert("America/New_York")
# Set Date col as index
df = df.set_index('date')
# Show df
df
```

Out[6]:

| date | Open | High | Low | Close | Volume | delta |
|---|---|---|---|---|---|---|
| **2010-01-03 12:00:00-05:00** | 1865.50 | 1867.50 | 1865.50 | 1867.00 | 251 | 125.500000 |
| **2010-01-03 12:01:00-05:00** | 1866.75 | 1867.00 | 1866.50 | 1866.50 | 76 | -76.000000 |
| **2010-01-03 12:02:00-05:00** | 1866.75 | 1867.50 | 1866.75 | 1866.75 | 151 | -151.000000 |
| **2010-01-03 12:03:00-05:00** | 1866.75 | 1867.00 | 1866.75 | 1867.00 | 26 | 26.000000 |
| **2010-01-03 12:04:00-05:00** | 1866.75 | 1866.75 | 1866.75 | 1866.75 | 1 | 0.000000 |
| **...** | ... | ... | ... | ... | ... | ... |
| **2026-06-25 23:07:00-04:00** | 29237.75 | 29240.75 | 29220.50 | 29230.25 | 212 | -7.851852 |
| **2026-06-25 23:08:00-04:00** | 29233.25 | 29244.75 | 29225.75 | 29237.25 | 194 | 40.842105 |
| **2026-06-25 23:09:00-04:00** | 29237.25 | 29239.50 | 29224.75 | 29228.25 | 163 | -85.644068 |
| **2026-06-25 23:10:00-04:00** | 29224.00 | 29253.75 | 29215.25 | 29253.75 | 290 | 290.000000 |
| **2026-06-25 23:11:00-04:00** | 29251.75 | 29254.25 | 29245.00 | 29248.50 | 136 | -33.081081 |

5465286 rows × 6 columns

In [7]:

```
class ORB_30(Strategy):
    SESSION_OPEN = "09:30"
    orb_minutes        = 30
    delta_threshold    = 0
    max_trades_per_day = 1

    def init(self):
        self._date        = None
        self._trades      = 0
        self._orb_high    = np.nan
        self._orb_low     = np.nan
        self._orb_delta   = 0.0
```

```
        self._range_ready = False

    def next(self):
        ts    = self.data.index[-1]
        today = ts.date()

        if today != self._date:
            self._date        = today
            self._trades      = 0
            self._orb_high    = np.nan
            self._orb_low     = np.nan
            self._orb_delta   = 0.0
            self._range_ready = False

        open_ts = pd.Timestamp(str(today) + " " + "09:30", tz=ts.tzinfo)
        orb_end = open_ts + pd.Timedelta(minutes=self.orb_minutes)

        if open_ts <= ts < orb_end:
            h, l = self.data.High[-1], self.data.Low[-1]
            self._orb_high  = h if np.isnan(self._orb_high) else max(self._orb_high
            self._orb_low   = l if np.isnan(self._orb_low)  else min(self._orb_low,
            self._orb_delta += self.data.df["delta"].iloc[-1]
            return

        if ts >= orb_end and not self._range_ready:
            self._range_ready = True

        if (not self._range_ready
                or self.position
                or self._trades >= self.max_trades_per_day
                or np.isnan(self._orb_high)):
            return

        close = self.data.Close[-1]

        # ── LONG: break above ORB high + positive delta ──────────────
        long_risk = close - self._orb_low
        if (close > self._orb_high
                and self._orb_delta > self.delta_threshold
                and long_risk > 0):

            tp = close + long_risk
            self.buy(sl=self._orb_low, tp=tp)
            self._trades += 1

        # ── SHORT: break below ORB low + negative delta ──────────────
        short_risk = self._orb_high - close
        if (close < self._orb_low
                and self._orb_delta < -self.delta_threshold
                and short_risk > 0):

            tp = close - short_risk
            self.sell(sl=self._orb_high, tp=tp)
            self._trades += 1
```

In [8]:

```
class ORB_15(Strategy):

    orb_minutes        = 15
    delta_threshold    = 0
    max_trades_per_day = 1

    def init(self):
        self._date        = None
        self._trades      = 0
        self._orb_high    = np.nan
        self._orb_low     = np.nan
        self._orb_delta   = 0.0
        self._range_ready = False

    def next(self):
        ts    = self.data.index[-1]
        today = ts.date()

        if today != self._date:
            self._date        = today
            self._trades      = 0
            self._orb_high    = np.nan
            self._orb_low     = np.nan
            self._orb_delta   = 0.0
            self._range_ready = False

        open_ts = pd.Timestamp(str(today) + " " + "09:30", tz=ts.tzinfo)
        orb_end = open_ts + pd.Timedelta(minutes=self.orb_minutes)

        if open_ts <= ts < orb_end:
            h, l = self.data.High[-1], self.data.Low[-1]
            self._orb_high  = h if np.isnan(self._orb_high) else max(self._orb_high
            self._orb_low   = l if np.isnan(self._orb_low)  else min(self._orb_low,
            self._orb_delta += self.data.df["delta"].iloc[-1]
            return

        if ts >= orb_end and not self._range_ready:
            self._range_ready = True

        if (not self._range_ready
                or self.position
                or self._trades >= self.max_trades_per_day
                or np.isnan(self._orb_high)):
            return

        close = self.data.Close[-1]

        # ── LONG: break above ORB high + positive delta ──────────────
        long_risk = close - self._orb_low
        if (close > self._orb_high
                and self._orb_delta > self.delta_threshold
                and long_risk > 0):

            tp = close + long_risk
            self.buy(sl=self._orb_low, tp=tp)
```

```
            self._trades += 1

        # ── SHORT: break below ORB low + negative delta ──────────────
        short_risk = self._orb_high - close
        if (close < self._orb_low
                and self._orb_delta < -self.delta_threshold
                and short_risk > 0):

            tp = close - short_risk
            self.sell(sl=self._orb_high, tp=tp)
            self._trades += 1
```

In [9]:
```
# Run the Back-Testing for 15 mins
bt_15 = Backtest(df, ORB_15, cash=100_000, exclusive_orders=True, trade_on_close=Tr
stats_15 = bt_15.run()
```

In [10]:
```
# Run the Back-Testing for 30 mins
bt_30 = Backtest(df, ORB_30, cash=100_000, exclusive_orders=True, trade_on_close=Tr
stats_30 = bt_30.run()
```

In [11]:
```
# Show Back-Testing data for 15 mins
stats_15
```

Out[11]:
```
Start                     2010-01-03 12:00...
End                       2026-06-25 23:11...
Duration                   6017 days 10:11:00
Exposure Time [%]                   12.79514
Equity Final [$]                  125183.67
Equity Peak [$]                   147773.42
Return [%]                         25.18367
Buy & Hold Return [%]             1466.60418
Return (Ann.) [%]                  1.06661
Volatility (Ann.) [%]              6.65255
CAGR [%]                           0.94508
Sharpe Ratio                       0.16033
Sortino Ratio                      0.22217
Calmar Ratio                       0.05097
Alpha [%]                         27.49436
Beta                              -0.00158
Max. Drawdown [%]                -20.92722
Avg. Drawdown [%]                 -0.29592
Max. Drawdown Duration     1380 days 13:59:00
Avg. Drawdown Duration        8 days 10:29:00
# Trades                                3321
Win Rate [%]                      51.24962
Best Trade [%]                     2.55967
Worst Trade [%]                   -6.63487
Avg. Trade [%]                     0.00718
Max. Trade Duration           7 days 04:18:00
Avg. Trade Duration           0 days 06:05:00
Profit Factor                      1.04157
Expectancy [%]                     0.00872
SQN                                0.78106
Kelly Criterion                    0.01783
_strategy                           ORB_15
_equity_curve                          ...
_trades                      Size  Entr...
dtype: object
```

In [12]:
```
# Show Back-Testing data for 30 mins
stats_30
```

Out[12]:
```
Start                     2010-01-03 12:00...
End                       2026-06-25 23:11...
Duration                   6017 days 10:11:00
Exposure Time [%]                   24.47411
Equity Final [$]                  102872.55
Equity Peak [$]                    116775.3
Return [%]                          2.87255
Buy & Hold Return [%]              1466.60418
Return (Ann.) [%]                   0.13386
Volatility (Ann.) [%]               8.56497
CAGR [%]                            0.11867
Sharpe Ratio                        0.01563
Sortino Ratio                       0.02149
Calmar Ratio                        0.00413
Alpha [%]                         -31.76587
Beta                                0.02362
Max. Drawdown [%]                   -32.429
Avg. Drawdown [%]                 -0.49989
Max. Drawdown Duration     5124 days 08:12:00
Avg. Drawdown Duration       21 days 19:16:00
# Trades                                3013
Win Rate [%]                       50.77995
Best Trade [%]                      3.88681
Worst Trade [%]                   -9.01801
Avg. Trade [%]                      0.00303
Max. Trade Duration          55 days 06:06:00
Avg. Trade Duration           0 days 12:58:00
Profit Factor                       1.02073
Expectancy [%]                      0.00585
SQN                                 0.08399
Kelly Criterion                     0.00205
_strategy                            ORB_30
_equity_curve                           ...
_trades                       Size  Entr...
dtype: object
```

In [13]:
```python
# Copy stats of 30mins backtesting
trades = stats_30._trades.copy()

# Size > 0 = Long, Size < 0 = short (backtesting.py convention)
trades["direction"] = np.where(trades["Size"] > 0, "Long", "Short")
trades["outcome"]   = np.where(trades["PnL"] > 0, "Win_TP",
                        np.where(trades["PnL"] < 0, "Loss_SL", "Breakeven"))

wins_long    = trades[(trades["outcome"] == "Win_TP")  & (trades["direction"] == "L
wins_short   = trades[(trades["outcome"] == "Win_TP")  & (trades["direction"] == "S
losses_long  = trades[(trades["outcome"] == "Loss_SL") & (trades["direction"] == "L
losses_short = trades[(trades["outcome"] == "Loss_SL") & (trades["direction"] == "S

print("Wins  Long :", len(wins_long))
print("Wins  Short:", len(wins_short))
print("Loss  Long :", len(losses_long))
print("Loss  Short:", len(losses_short))
```

```
Wins  Long : 874
Wins  Short: 656
Loss  Long : 771
Loss  Short: 712
```

In [14]:
```python
# Winning trades review ORB 30min
trades = stats_30._trades.copy()
trades["direction"] = np.where(trades["Size"] > 0, "Long", "Short")
trades["outcome"]   = np.where(trades["PnL"] > 0, "Win", "Loss")

def clean_review(df):
    return pd.DataFrame({
        "Entry Date"  : pd.to_datetime(df["EntryTime"]).dt.strftime("%Y-%m-%d"),
        "Entry Time"  : pd.to_datetime(df["EntryTime"]).dt.strftime("%H:%M"),
        "Exit Date"   : pd.to_datetime(df["ExitTime"]).dt.strftime("%Y-%m-%d"),
        "Exit Time"   : pd.to_datetime(df["ExitTime"]).dt.strftime("%H:%M"),
        "Entry Price" : df["EntryPrice"].round(2),
        "Stop Loss"   : df["SL"].round(2),
        "Take Profit" : df["TP"].round(2),
        "PnL $"       : df["PnL"].round(2),
        "Return %"    : (df["ReturnPct"] * 100).round(3),
    }).sort_values("Entry Date", ascending=False).reset_index(drop=True)

wins_long  = trades[(trades["outcome"] == "Win") & (trades["direction"] == "Long")]
wins_short = trades[(trades["outcome"] == "Win") & (trades["direction"] == "Short")]

review_long  = clean_review(wins_long)
review_short = clean_review(wins_short)

print(f"Winning LONG  trades: {len(review_long)}")
print(f"Winning SHORT trades: {len(review_short)}")
```

```
Winning LONG  trades: 874
Winning SHORT trades: 656
```

In [15]:
```python
# Copy stats of 15mins backtesting
trades = stats_15._trades.copy()

# Size > 0 = long, Size < 0 = short (backtesting.py convention)
trades["direction"] = np.where(trades["Size"] > 0, "Long", "Short")
trades["outcome"]   = np.where(trades["PnL"] > 0, "Win_TP",
                          np.where(trades["PnL"] < 0, "Loss_SL", "Breakeven"))

wins_long    = trades[(trades["outcome"] == "Win_TP")  & (trades["direction"] == "L
wins_short   = trades[(trades["outcome"] == "Win_TP")  & (trades["direction"] == "S
losses_long  = trades[(trades["outcome"] == "Loss_SL") & (trades["direction"] == "L
losses_short = trades[(trades["outcome"] == "Loss_SL") & (trades["direction"] == "S

print("Wins  Long :", len(wins_long))
print("Wins  Short:", len(wins_short))
print("Loss  Long :", len(losses_long))
print("Loss  Short:", len(losses_short))
```

```
Wins  Long : 938
Wins  Short: 764
Loss  Long : 852
Loss  Short: 767
```

In [16]:
```
# Winning trades review ORB 15min
trades = stats_15._trades.copy()
trades["direction"] = np.where(trades["Size"] > 0, "Long", "Short")
trades["outcome"]   = np.where(trades["PnL"] > 0, "Win", "Loss")

def clean_review(df):
    return pd.DataFrame({
        "Entry Date"  : pd.to_datetime(df["EntryTime"]).dt.strftime("%Y-%m-%d"),
        "Entry Time"  : pd.to_datetime(df["EntryTime"]).dt.strftime("%H:%M"),
        "Exit Date"   : pd.to_datetime(df["ExitTime"]).dt.strftime("%Y-%m-%d"),
        "Exit Time"   : pd.to_datetime(df["ExitTime"]).dt.strftime("%H:%M"),
        "Entry Price" : df["EntryPrice"].round(2),
        "Stop Loss"   : df["SL"].round(2),
        "Take Profit" : df["TP"].round(2),
        "PnL $"       : df["PnL"].round(2),
        "Return %"    : (df["ReturnPct"] * 100).round(3),
    }).sort_values("Entry Date", ascending=False).reset_index(drop=True)

wins_long  = trades[(trades["outcome"] == "Win") & (trades["direction"] == "Long")]
wins_short = trades[(trades["outcome"] == "Win") & (trades["direction"] == "Short")]

review_long  = clean_review(wins_long)
review_short = clean_review(wins_short)

print(f"Winning LONG  trades: {len(review_long)}")
print(f"Winning SHORT trades: {len(review_short)}")
```

```
Winning LONG  trades: 938
Winning SHORT trades: 764
```